

ipc_assignments_summary.md

IPC (Semaphore) Assignments — সব সমাধান একসাথে

1. Section A2 — তিনটা থ্রেড, একসাথে A, B, C থাকলে newline

Pattern Definition

তিনটা থ্রেড আলাদাভাবে অসীম লুপে যথাক্রমে A, B, C প্রিন্ট করবে। যখনই একটা লাইনে অন্তত একবার করে A, B, এবং C — তিনটাই এসে যাবে, তখন একটা newline প্রিন্ট হবে। মোট N-টা লাইন প্রিন্ট হওয়া পর্যন্ত চলবে। শুধু ১টা semaphore দিয়েই যথেষ্ট।

Pattern (উদাহরণ, N = 3)

```
ABC
BABABAC
CACACACACAB
```

Solution Definition

একটা mutex semaphore দিয়ে তিনটা shared flag (sawA, sawB, sawC) protect করা হয়েছে — যখনই তিনটাই true হয়, newline প্রিন্ট করে flag গুলো reset করে দেওয়া হয়।

Full Code

cpp

```
#include <bits/stdc++.h>
#include <pthread.h>
#include <semaphore.h>
#include <sched.h>
using namespace std;

sem_t mutexSem;          // ekmatro semaphore, mutex hisebe
int N;
int linesPrinted = 0;
bool sawA = false, sawB = false, sawC = false;

void* threadA(void* arg) {
    while (true) {
        sem_wait(&mutexSem);
        if (linesPrinted >= N) { sem_post(&mutexSem); break; }

        cout << 'A';
        cout.flush();
        sawA = true;

        if (sawA && sawB && sawC) {
            cout << '\n';
            sawA = sawB = sawC = false;
        }
    }
}
```

```

        linesPrinted++;
    }
    sem_post(&mutexSem);
    sched_yield();
}
return nullptr;
}

void* threadB(void* arg) {
    while (true) {
        sem_wait(&mutexSem);
        if (linesPrinted >= N) { sem_post(&mutexSem); break; }

        cout << 'B';
        cout.flush();
        sawB = true;

        if (sawA && sawB && sawC) {
            cout << '\n';
            sawA = sawB = sawC = false;
            linesPrinted++;
        }
        sem_post(&mutexSem);
        sched_yield();
    }
    return nullptr;
}

void* threadC(void* arg) {
    while (true) {
        sem_wait(&mutexSem);
        if (linesPrinted >= N) { sem_post(&mutexSem); break; }

        cout << 'C';
        cout.flush();
        sawC = true;

        if (sawA && sawB && sawC) {
            cout << '\n';
            sawA = sawB = sawC = false;
            linesPrinted++;
        }
        sem_post(&mutexSem);
        sched_yield();
    }
    return nullptr;
}

int main() {
    cout << "Enter N: ";
    cin >> N;

    sem_init(&mutexSem, 0, 1);

    pthread_t t1, t2, t3;
    pthread_create(&t1, nullptr, threadA, nullptr);
    pthread_create(&t2, nullptr, threadB, nullptr);
    pthread_create(&t3, nullptr, threadC, nullptr);

    pthread_join(t1, nullptr);
    pthread_join(t2, nullptr);
    pthread_join(t3, nullptr);

    sem_destroy(&mutexSem);
    return 0;
}

```

`sched_yield()` শুধু fairness-এর জন্য — একই thread যাতে বারবার lock জিতে না যায়, এতে আসল correctness

বদলায় না।

2. Section B1 — Underscore/Plus পিরামিড

Pattern Definition

দুইটা থ্রেড: একটা `_` প্রিন্ট করে, একটা `+`। `N` ইনপুট নিয়ে এমনভাবে প্রিন্ট করতে হবে যাতে প্রতি লাইনে underscore কমতে থাকে আর plus বাড়তে থাকে, মোট প্রতি লাইনে যোগফল সবসময় `N`।

Pattern (N = 10)

```
_____+
_______++
______+++
_____++++
____+++++
```

Solution Definition

দুইটা counting semaphore (`semA`, `semB`) দিয়ে strict ping-pong — A একবার print করে B-কে জাগায়, B print করে A-কে জাগায়। কোনো busy-wait নাই, দুইজনই সত্যিকারের block করে থাকে।

Full Code

cpp

```
#include <bits/stdc++.h>
#include <pthread.h>
#include <semaphore.h>
using namespace std;

sem_t semA, semB; // dujon-er jonno dujon semaphore
int N;
int turnsLeft;    // mot koyta line baki

void* threadA(void* arg) {
    int underscoreCount = N - 1;
    for (int i = 0; i < turnsLeft; i++) {
        sem_wait(&semA); // B na dakle ekhane block
        for (int j = 0; j < underscoreCount; j++) cout << '_';
        underscoreCount--;
        sem_post(&semB); // B-ke jagay
    }
    return nullptr;
}

void* threadB(void* arg) {
    int plusCount = 1;
    for (int i = 0; i < turnsLeft; i++) {
        sem_wait(&semB); // A na dakle ekhane block
        for (int j = 0; j < plusCount; j++) cout << '+';
        cout << '\n';
        plusCount++;
        sem_post(&semA); // A-ke abar jagay
    }
    return nullptr;
}
```

```
int main() {
    cout << "Enter N: ";
    cin >> N;
    turnsLeft = N / 2;

    sem_init(&semA, 0, 1);    // A age jabe
    sem_init(&semB, 0, 0);    // B block thakbe

    pthread_t t1, t2;
    pthread_create(&t1, nullptr, threadA, nullptr);
    pthread_create(&t2, nullptr, threadB, nullptr);

    pthread_join(t1, nullptr);
    pthread_join(t2, nullptr);

    sem_destroy(&semA);
    sem_destroy(&semB);
    return 0;
}
```

দুই thread-ই একই turnsLeft সংখ্যকবার loop চালায় বলে odd/even N — দুই ক্ষেত্রেই ঠিক থামে।

3. Section B2 — N লাইন, প্রতি লাইনে L অক্ষর, শেষে \$

Pattern Definition

A, B, C তিনটা থ্রেড অসীম লুপে print করবে। প্রতি লাইনে মোট Lটা letter জমা হলে \$ প্রিন্ট হবে। এভাবে Nটা লাইন হবে।

Pattern (N = 3, L = 10)

```
ABCBCBCBAC$
CBACABCACB$
BACBCABABA$
```

Solution Definition

একটা mutex semaphore দিয়ে shared letterCount (লাইনে এখন পর্যন্ত কতগুলো অক্ষর) আর lineCount (কয়টা লাইন শেষ) ট্র্যাক করা হয়েছে; একটা lastPrinted variable দিয়ে একই letter পরপর না আসাটা নিশ্চিত করা হয়েছে (fair interleaving-এর জন্য)।

Full Code

cpp

```
#include <bits/stdc++.h>
#include <pthread.h>
#include <semaphore.h>
#include <sched.h>
using namespace std;

sem_t mutexSem;
int N, L;
int letterCount = 0;
int lineCount = 0;
char lastPrinted = 0;    // fair interleaving: same letter porpor na asha nishchit kore
```

```
void* threadA(void* arg) {
    while (true) {
        sem_wait(&mutexSem);
        if (lineCount == N) { sem_post(&mutexSem); break; }
        if (lastPrinted == 'A') { sem_post(&mutexSem); sched_yield(); continue; }

        cout << 'A';
        lastPrinted = 'A';
        letterCount++;
        if (letterCount == L) {
            letterCount = 0;
            lineCount++;
            cout << "$" << endl;
        }
        sem_post(&mutexSem);
        sched_yield();
    }
    return nullptr;
}

void* threadB(void* arg) {
    while (true) {
        sem_wait(&mutexSem);
        if (lineCount == N) { sem_post(&mutexSem); break; }
        if (lastPrinted == 'B') { sem_post(&mutexSem); sched_yield(); continue; }

        cout << 'B';
        lastPrinted = 'B';
        letterCount++;
        if (letterCount == L) {
            letterCount = 0;
            lineCount++;
            cout << "$" << endl;
        }
        sem_post(&mutexSem);
        sched_yield();
    }
    return nullptr;
}

void* threadC(void* arg) {
    while (true) {
        sem_wait(&mutexSem);
        if (lineCount == N) { sem_post(&mutexSem); break; }
        if (lastPrinted == 'C') { sem_post(&mutexSem); sched_yield(); continue; }

        cout << 'C';
        lastPrinted = 'C';
        letterCount++;
        if (letterCount == L) {
            letterCount = 0;
            lineCount++;
            cout << "$" << endl;
        }
        sem_post(&mutexSem);
        sched_yield();
    }
    return nullptr;
}

int main() {
    cout << "Enter N and L: ";
    cin >> N >> L;

    sem_init(&mutexSem, 0, 1);

    pthread_t t1, t2, t3;
    pthread_create(&t1, nullptr, threadA, nullptr);
    pthread_create(&t2, nullptr, threadB, nullptr);
    pthread_create(&t3, nullptr, threadC, nullptr);
}
```

```
pthread_join(t1, nullptr);
pthread_join(t2, nullptr);
pthread_join(t3, nullptr);

sem_destroy(&mutexSem);
return 0;
}
```

4. Section B2 (Batch 22 variant) — P, Q, R বাড়তে থাকা quota

Pattern Definition

তিনটা থ্রেড P, Q, R প্রিন্ট করবে। iteration k-এ প্রতিটা letter ঠিক k বার আসবে (random order-এ), তারপর newline। মোট N iteration নেওয়া হবে ইনপুট হিসেবে।

Pattern (N = 3)

```
PQR
QQPRRP
PQPQRQRRP
```

Solution Definition

target variable প্রতি iteration-এ বাড়ে (১, ২, ৩...); প্রতিটা letter-এর নিজের counter (countP/Q/R) target-এ পৌঁছালে সেই letter আর print হয় না, বাকিদের জন্য wait করে। তিনজনই target ছুঁলে newline আর পরের iteration শুরু।

Full Code

```
cpp

#include <bits/stdc++.h>
#include <pthread.h>
#include <semaphore.h>
#include <sched.h>
using namespace std;

sem_t mutexSem;
int N;
int target = 1;
int countP = 0, countQ = 0, countR = 0;
int iterationsDone = 0;

void* threadP(void* arg) {
    while (true) {
        sem_wait(&mutexSem);
        if (iterationsDone == N) { sem_post(&mutexSem); break; }

        if (countP < target) { cout << 'P'; countP++; }

        if (countP == target && countQ == target && countR == target) {
            cout << '\n';
            countP = countQ = countR = 0;
            iterationsDone++;
            target++;
        }
    }
}
```

```

        sem_post(&mutexSem);
        sched_yield();
    }
    return nullptr;
}

void* threadQ(void* arg) {
    while (true) {
        sem_wait(&mutexSem);
        if (iterationsDone == N) { sem_post(&mutexSem); break; }

        if (countQ < target) { cout << 'Q'; countQ++; }

        if (countP == target && countQ == target && countR == target) {
            cout << '\n';
            countP = countQ = countR = 0;
            iterationsDone++;
            target++;
        }
        sem_post(&mutexSem);
        sched_yield();
    }
    return nullptr;
}

void* threadR(void* arg) {
    while (true) {
        sem_wait(&mutexSem);
        if (iterationsDone == N) { sem_post(&mutexSem); break; }

        if (countR < target) { cout << 'R'; countR++; }

        if (countP == target && countQ == target && countR == target) {
            cout << '\n';
            countP = countQ = countR = 0;
            iterationsDone++;
            target++;
        }
        sem_post(&mutexSem);
        sched_yield();
    }
    return nullptr;
}

int main() {
    cout << "Enter number of iterations: ";
    cin >> N;

    sem_init(&mutexSem, 0, 1);

    pthread_t t1, t2, t3;
    pthread_create(&t1, nullptr, threadP, nullptr);
    pthread_create(&t2, nullptr, threadQ, nullptr);
    pthread_create(&t3, nullptr, threadR, nullptr);

    pthread_join(t1, nullptr);
    pthread_join(t2, nullptr);
    pthread_join(t3, nullptr);

    sem_destroy(&mutexSem);
    return 0;
}

```

5. Section C1 (Batch 22) — চারটা থ্রেড, ফিক্সড quota, busy-wait ছাড়া

Pattern Definition

A, B, C তিনটা থ্রেড infinite loop-এ print করবে; প্রতি লাইনে ঠিক 3টা A, 4টা B, 5টা C (মোট ১২টা) থাকতে হবে। ৪র্থ থ্রেড ১২টা letter হয়ে গেলে newline প্রিন্ট করবে। মোট ৭টা লাইন। শর্ত: **busy-waiting এড়াতে হবে, ৪টা counting semaphore লাগবে।**

Pattern (উদাহরণ)

```
AAABBBBCCCCC
ABCBABCBACACC
BACABCBACBCC
```

Solution Definition

প্রতি লাইনের জন্য আলাদা quota-semaphore (semA=3, semB=4, semC=5) — quota শেষ হলে thread সত্যিকারের block হয়ে থাকে। ৪র্থ semaphore (semPrinted) দিয়ে ৪র্থ থ্রেড ঠিক ১২টা signal গোলেন, newline print করে পরের লাইনের quota আবার refill করে দেয়।

Full Code

cpp

```
#include <bits/stdc++.h>
#include <pthread.h>
#include <semaphore.h>
using namespace std;

sem_t semA, semB, semC, semPrinted;

int N;
const int perLineA = 3, perLineB = 4, perLineC = 5;
const int perLineTotal = perLineA + perLineB + perLineC; // 12

void* threadA(void* arg) {
    while (true) {
        sem_wait(&semA);          // quota shesh hole ekhane block (busy-wait na)
        cout << 'A';
        sem_post(&semPrinted);
    }
    return nullptr;
}

void* threadB(void* arg) {
    while (true) {
        sem_wait(&semB);
        cout << 'B';
        sem_post(&semPrinted);
    }
    return nullptr;
}

void* threadC(void* arg) {
    while (true) {
        sem_wait(&semC);
        cout << 'C';
        sem_post(&semPrinted);
    }
    return nullptr;
}

void* threadD(void* arg) { // newline printer + refill controller
    for (int line = 0; line < N; line++) {
```



```

        for (int i = 0; i < perLineTotal; i++) {
            sem_wait(&semPrinted);    // ei line-er 12 ta letter howa porjonto block
        }
        cout << '\n';

        if (line != N - 1) {          // shesh line hole ar refill lagbe na
            for (int i = 0; i < perLineA; i++) sem_post(&semA);
            for (int i = 0; i < perLineB; i++) sem_post(&semB);
            for (int i = 0; i < perLineC; i++) sem_post(&semC);
        }
    }
    return nullptr;
}

int main() {
    cout << "Enter number of lines: ";
    cin >> N;

    sem_init(&semA, 0, perLineA);
    sem_init(&semB, 0, perLineB);
    sem_init(&semC, 0, perLineC);
    sem_init(&semPrinted, 0, 0);

    pthread_t tA, tB, tC, tD;
    pthread_create(&tA, nullptr, threadA, nullptr);
    pthread_create(&tB, nullptr, threadB, nullptr);
    pthread_create(&tC, nullptr, threadC, nullptr);
    pthread_create(&tD, nullptr, threadD, nullptr);

    // N line shesh howar por A/B/C chirokal block thakbe -> tai detach, join na kore
    pthread_detach(tA);
    pthread_detach(tB);
    pthread_detach(tC);

    pthread_join(tD, nullptr);

    sem_destroy(&semA);
    sem_destroy(&semB);
    sem_destroy(&semC);
    sem_destroy(&semPrinted);
    return 0;
}

```

গুরুত্বপূর্ণ: প্রতি লাইনের quota আলাদাভাবে refill করা হয়েছে (পুরো প্রোগ্রামের জন্য একবারে দিলে per-line-এ $3A+4B+5C$ mixture নিশ্চিত করা যায় না), আর শেষ লাইনের পর refill বন্ধ করা হয়েছে (নাহলে একটা অতিরিক্ত অসম্পূর্ণ লাইন প্রিন্ট হয়ে যেত)।